



Draw It or Lose It
CS 230 Project Software Design Document
Version 3.0

Table of Contents

CS 230 Project Software Design Template	1
Table of Contents	2
Document Revision History	2
Executive Summary	3
Requirements	3
Design Constraints	3
System Architecture View	3
Domain Model	3
Evaluation	4
Recommendations	7

Document Revision History

Version	Date	Author	Comments
1.0	11/16/2025	Alec Brandt	Initial draft of software design document for "Draw It or Lose It" web-based application.
2.0	12/7/2025	Alec Brandt	Revised for Project Two: Platform and development tools evaluation for multi-platform expansion. Includes licensing costs.
3.0	12/14/2025	Alec Brandt	Expanded on the recommendations section significantly and refined my communication so it's clear and consistent.

Executive Summary

This document outlines the proposed software design for "Draw It or Lose It," a web-based game application for our client, The Gaming Room. Currently, "Draw It or Lose It" exists solely as an Android application. The objective of this project is to expand its availability to multiple platforms through a web-based distributed environment. This design focuses on streamlining development, addressing key software requirements, and ensuring a robust, scalable, and secure application. Critical information for The Gaming Room to proceed includes understanding the proposed architecture, the object-oriented principles applied, and the evaluation of various operating platforms and development tools to ensure a smooth transition and broad accessibility for their game. This document will serve as a foundational guide for the development process, laying out the technical approach to meet the client's vision for a multi-platform gaming experience. The evaluation and recommendations sections specifically address platform suitability, development considerations, and technical requirements necessary for this expansion.

Requirements

The primary business requirement is to expand "Draw It or Lose It" from an Android-only app to a web-based game serving multiple platforms. Technical requirements include enabling games with one or more teams, each having multiple players. Game and team names must be unique for user validation. Crucially, only one instance of the game's core service can exist in memory at any given time, achieved through unique identifiers for games, teams, and players.

Design Constraints

Developing "Draw It or Lose It" as a web-based, distributed application imposes several design constraints. Performance will be critical, necessitating efficient data transfer and client-side rendering to ensure a smooth user experience across various devices and network conditions. Scalability is another constraint; the system must handle a growing number of concurrent users and games without degradation. Security must be a foundational aspect, protecting user data and preventing unauthorized access in a distributed environment. Interoperability across diverse platforms (desktop browsers, mobile browsers) requires careful consideration of web standards and responsive design. These constraints imply the need for a robust backend, optimized database interactions, and a well-structured frontend capable of adapting to different screen sizes and input methods.

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

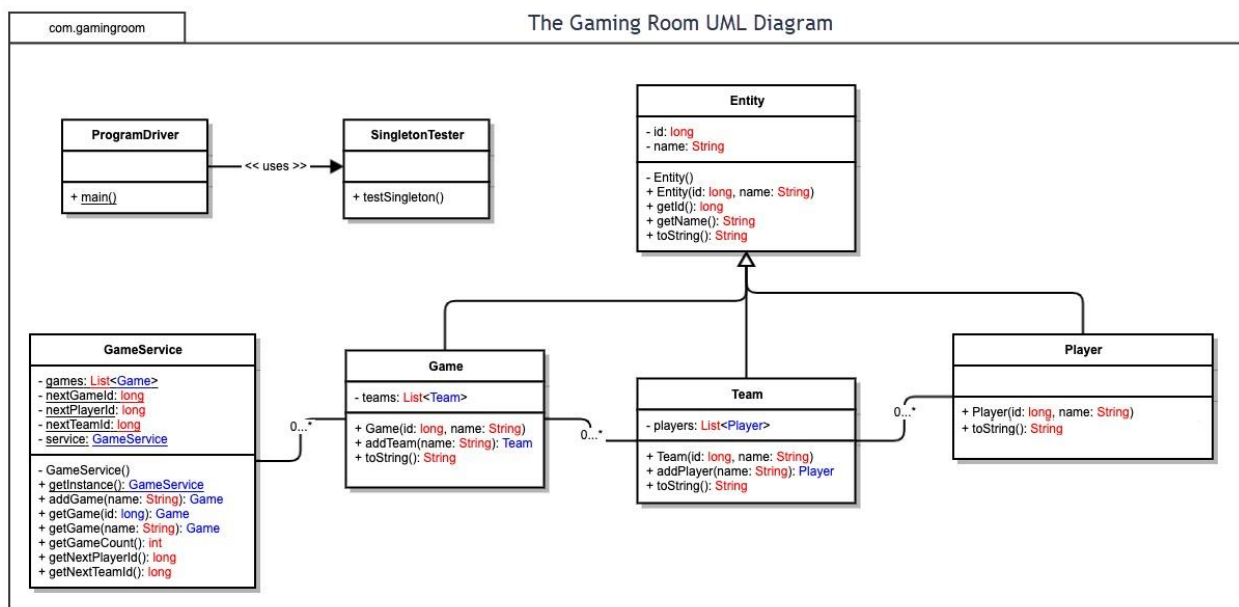
Domain Model

The provided UML class diagram for "The Gaming Room" illustrates the core components and their relationships within the application. The Entity class serves as a base class, abstracting common attributes (id and name) and behaviors for all major entities. The Game, Team, and Player classes inherit

from Entity, demonstrating the object-oriented principle of inheritance, which promotes code reuse and a clear hierarchy.

The GameService class implements the Singleton pattern, ensuring that only one instance of the game service exists at any given time, which fulfills the requirement of having a single point of control for managing games, teams, and players. This is crucial for maintaining consistent state and generating unique identifiers (nextGameId, nextPlayerId, nextTeamId).

The Game class aggregates Team objects, and Team aggregates Player objects, representing one-to-many relationships. The GameService methods like addGame() and getGame(), and similarly for Team and Player operations, leverage the Iterator pattern to ensure the uniqueness of game and team names, allowing for efficient searching and validation before adding new instances. This adherence to object-oriented principles ensures the software requirements are met efficiently by providing a structured, maintainable, and scalable design for the "Draw It or Lose It" application.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	Mac is generally not ideal for server-side hosting of web-based applications due to higher hardware costs and a smaller server market share. While technically capable, its ecosystem is less optimized for large-scale enterprise server deployments compared to Linux or Windows Server, leading to potential disadvantages in terms of cost-effectiveness and specialized support. Licensing costs: \$500-\$999 per server plus Apple support fees.	Linux offers significant advantages for hosting web-based applications, including high stability, security, cost-effectiveness (open-source), and extensive community support. Its weaknesses are a steeper learning curve for new administrators and potential compatibility issues with proprietary software, though widely supported by web technologies. Licensing costs: \$0 for OS (open-source); optional support contracts \$500-\$2000/year.	Windows Server provides robust hosting for web applications, especially those built with Microsoft technologies like .NET. Advantages include strong tooling, ease of management for Windows-centric teams, and good enterprise support. Weaknesses include higher licensing costs compared to Linux and potentially greater resource consumption. Licensing costs: \$500-\$6000 per server annually, plus SQL Server licensing (\$3,717+/year).	Mobile devices are client-side platforms, not typically used for server-side hosting of web applications. Their characteristics (limited processing power, battery life, network dependency) make them unsuitable for this role. Hosting on mobile devices would introduce severe performance, reliability, and security weaknesses.

Client Side	Software development for Mac clients (browsers) involves considerations for UI/UX consistency with macOS standards. Cost for development can be higher due to specific Apple hardware and software requirements. Time is comparable to other desktop platforms, and expertise in web technologies (HTML, CSS, JavaScript) is transferable, with additional focus on Safari browser compatibility and performance.	Development for Linux clients (browsers) focuses on compatibility with various distributions and desktop environments. Cost is generally lower due to open-source tools. Time is comparable, and expertise in standard web technologies is sufficient. Testing across different Linux browser versions and environments is important to ensure broad compatibility.	Development for Windows clients (browsers) is widespread, leveraging familiar tools. Cost is moderate, with readily available development environments. Time is standard, and expertise in web technologies is directly applicable. Ensuring cross-browser compatibility across Edge, Chrome, and Firefox on Windows is a key consideration.	Development for mobile devices requires significant considerations for responsive design, touch interactions, and performance optimization for varying screen sizes and network speeds. Costs can be higher due to the need for extensive testing on diverse devices and operating systems (iOS, Android). Time is often longer due to complexities of mobile web development, and specialized expertise in mobile-first design and performance tuning is crucial.
--------------------	--	--	---	---

Development Tools	For Mac deployment, relevant programming languages include JavaScript, Python, Ruby, PHP, Java, etc., supported by IDEs like VS Code, IntelliJ IDEA, Sublime Text, and Safari's developer tools. Build tools like Webpack, Gulp, and package managers like npm/Yarn are also used. Licensing costs: VS Code (free), IntelliJ IDEA (\$599/year), Sublime Text (\$99). Team impact: 1-2 front-end developers with macOS/Safari expertise needed.	For Linux deployment, popular languages include JavaScript, Python, Ruby, PHP, Java, Go, etc. IDEs like VS Code, Sublime Text, Vim, and IntelliJ IDEA are commonly used. Build systems (Maven, Gradle), containerization (Docker), and version control (Git) are essential for efficient development and deployment on Linux servers. Licensing costs: Mostly free/open-source; IntelliJ IDEA (\$599/year) optional. Team impact: 1-2 backend developers with Linux/DevOps skills required.	For Windows deployment, languages such as JavaScript, C#, Python, Java, and PHP are relevant. IDEs like Visual Studio, VS Code, and IntelliJ IDEA are widely used. Tools for .NET development are particularly strong. Package managers (npm, NuGet), build tools (MSBuild, Webpack), and version control (Git) are standard. Licensing costs: Visual Studio Community (free), Professional (\$1,199/year), VS Code (free). Team impact: 1-2 backend developers with C#/.NET expertise; licensing per developer increases costs.	For mobile device deployment (web-based), primary languages are HTML, CSS, and JavaScript. Frameworks like React Native (for hybrid apps, though less relevant for purely web-based), or responsive design frameworks like Bootstrap/Tailwind CSS are used. IDEs like VS Code, along with mobile browser developer tools, emulators, and debugging tools, are critical. Licensing costs: All tools free/open-source. Team impact: 2-3 developers needed (responsive design specialists + QA for iOS/Android testing).
--------------------------	--	---	--	---

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform:** For "The Gaming Room" to expand "Draw It or Lose It" to other computing environments, a web-based platform architecture is recommended, with Linux as the server-side operating system and modern web technologies for client applications. The server-side will run on Linux-based environments within containerized systems, while clients will access the application through web browsers on Windows, macOS, Linux, iOS, and Android devices. This recommendation is based on several key factors. Linux is the ideal server operating system because it offers superior stability, security, and cost-effectiveness compared to Windows Server or Mac. As an open-source platform, Linux eliminates licensing costs and provides extensive community support for web application development. Additionally, Linux's

architecture is optimized for multi-user, multitasking environments, making it ideal for hosting multiple concurrent game instances. The web-based client approach maximizes accessibility since any device with a modern web browser (Chrome, Firefox, Safari, Edge) can run the application without requiring separate native development for each platform. This strategy reduces development time and maintenance costs while ensuring broad market reach and consistent user experience across all platforms.

2. **Operating Systems Architectures:** The recommended architecture for "Draw It or Lose It" employs a three-tier, cloud-based system deployed on Linux servers within containerized environments. The first tier is the client-side presentation layer, which runs in web browsers across all operating platforms. This layer uses HTML5, CSS3, and JavaScript frameworks to deliver a responsive user interface that adapts to desktop browsers (Windows, macOS, Linux) and mobile browsers (iOS Safari, Android Chrome). The presentation layer communicates with the backend through standard web protocols and handles all user interactions, game rendering, and local state management. The second tier is the application layer, which runs on Linux servers using Docker containers orchestrated by Kubernetes. This layer handles the core game logic, manages game instances, processes player actions, and coordinates real-time communication between players. Each container runs independently but shares system resources efficiently, and Kubernetes automatically manages scaling, load balancing, and failover. This architecture ensures that multiple concurrent games can run simultaneously without interfering with each other, satisfying the requirement that only one game service instance exists in memory at any given time through proper service isolation and containerization. The third tier is the data layer, consisting of databases and object storage services. The relational aspects of games, teams, and players are stored in a NoSQL database, while static assets like the stock drawings library are stored in cloud object storage. Linux's superior handling of multi-process scenarios and file system efficiency makes it the optimal choice for managing these multiple tiers and ensuring smooth communication between layers. This multi-tier approach provides scalability, reliability, and maintainability while allowing The Gaming Room to expand their player base without significant infrastructure changes.
3. **Storage Management:** An effective storage management strategy for "Draw It or Lose It" must address both the dynamic game data and the large static image library. For dynamic data, a NoSQL document database such as MongoDB or Amazon DynamoDB is recommended. This database will store game state information, player profiles, team records, and game history. NoSQL databases are ideal for this application because game data structures may evolve over time, and document-based storage provides the flexibility to add new fields without requiring schema migrations. The database should implement automatic backup and replication across multiple geographic regions to ensure data durability and availability. For the static asset library of approximately 1.6 gigabytes of stock drawings, cloud object storage such as Amazon S3, Azure Blob Storage, or Google Cloud Storage is the appropriate choice. These services offer several advantages specifically relevant to the game's requirements. First, they provide high availability and durability, with automatic redundancy ensuring that images remain accessible even if individual storage devices fail. Second, they support content delivery through Content Delivery Networks (CDNs), which cache images at locations geographically closer to users, reducing latency when images need to be retrieved during gameplay. Third, they are cost-effective for storing large volumes of static data. The storage management strategy should also implement intelligent caching at the application layer, where frequently accessed images are cached in memory during active games to minimize repeated retrieval from cloud storage.

Game data should be organized with proper indexing on game IDs and player IDs to enable rapid lookups when players join games or new rounds begin. Version control of game state snapshots should be maintained to support game replay functionality and dispute resolution if needed.

4. **Memory Management:** Memory management in the recommended cloud-based Linux environment operates at multiple levels to ensure efficient performance during rapid image cycling and concurrent game support. At the infrastructure level, memory is managed by the containerized environment and orchestration system. When the "Draw It or Lose It" application runs in a Docker container on Linux, the container is allocated a specific amount of memory, typically between 512 MB and 2 GB depending on expected concurrent user load. Kubernetes monitors memory usage and automatically scales containers horizontally if memory pressure becomes too high, spawning additional container instances to distribute the load. This automated scaling ensures that the application can handle growth in concurrent games without degradation. At the application level, the game software should implement efficient memory management strategies specific to the drawing image library. Since the application must render images at a steady rate with complete images available at the 30-second mark, implementing a caching strategy is critical. The application should maintain an in-memory cache of approximately 100 to 150 megabytes that holds the most frequently used images and recently accessed images. This prevents repeated loading from cloud storage during active gameplay. Additionally, the application should implement object pooling, where image objects are reused rather than created and destroyed for each game round. This reduces the garbage collection overhead that would otherwise occur with Java's automatic memory management. The Java Virtual Machine should be configured with appropriate garbage collection parameters to minimize pause times during critical gameplay moments. For the multi-game scenario, each game instance should be isolated in its own memory space or container to prevent one poorly-behaving game from consuming excessive memory and impacting other games. When players join a game, the necessary image assets should be preloaded during the joining process rather than waiting until the game begins, ensuring smooth performance once gameplay starts.
5. **Distributed Systems and Networks:** Communication between the web-based client application and the Linux backend servers in "Draw It or Lose It" must accommodate both event-driven interactions and real-time game synchronization. The recommended approach uses two complementary communication protocols. RESTful APIs over HTTPS handle stateless requests for operations such as user authentication, game creation, joining existing games, and retrieving static game metadata. For example, when a player wants to join a game, the client sends an HTTP POST request to the backend API endpoint, which validates the player information, updates the game state in the database, and returns confirmation. These REST calls are appropriate for operations that don't require continuous connection and can tolerate slight latencies. However, for real-time gameplay interactions, WebSocket connections are essential. WebSockets maintain a persistent, bidirectional communication channel between each client and the backend, allowing real-time updates for critical game events. When a drawing is being rendered, the backend streams updates through the WebSocket as new drawing data arrives, ensuring all team members see the image develop in real time. Timer synchronization is also critical, and the backend sends timer updates through WebSockets to ensure all players see the same countdown, preventing disputes about whether guesses were submitted before time expired. Dependencies in this distributed system include reliable internet connectivity for all clients. To mitigate the impact of network outages, the application should implement robust error handling and automatic reconnection logic. If a player's WebSocket connection drops, the

client should attempt to reconnect automatically, and the backend should maintain game state so that the player can rejoin without losing progress. Client-side caching of game state allows the interface to remain responsive even during brief network interruptions. Additionally, Content Delivery Networks should be used to serve static assets like the stock drawings library from servers geographically distributed worldwide, minimizing latency for users in different regions and reducing the load on the primary backend servers.

6. **Security:** Security in a distributed multi-platform game application like "Draw It or Lose It" requires protection at multiple layers. User data, including player names, team affiliations, and game history, must be protected both during transmission and while at rest. All communication between clients and the backend will use HTTPS with TLS encryption, ensuring that usernames, passwords, and game data cannot be intercepted by attackers on the network. User authentication will be implemented using industry-standard OAuth 2.0 or OpenID Connect protocols. Rather than storing passwords directly, the application will integrate with identity providers, reducing the risk of credential breaches. This approach also allows players to use single sign-on with existing accounts from providers like Google or Microsoft. Once authenticated, each user session will be assigned a secure session token that must be included in subsequent requests; tokens should have expiration times to limit the window of opportunity if a token is compromised. Authorization controls must prevent unauthorized game access. Only players who have been explicitly added to a team should be able to participate in that team's games, and this authorization should be verified on every sensitive operation. Player data stored in the NoSQL database should be encrypted at rest using AES-256 encryption or equivalent, protecting against unauthorized access to the database itself. The cloud infrastructure's built-in security features, such as firewalls and network access control lists, should be configured to restrict direct database access and allow communication only through the application backend. Additional security measures include input validation on all client submissions to prevent injection attacks, where a malicious player might attempt to inject code through text fields. Regular security audits and penetration testing should be performed to identify vulnerabilities before they can be exploited. Rate limiting should be implemented to prevent denial-of-service attacks where an attacker repeatedly attempts to create games or submit guesses to overload the system. Finally, the application should log all security-relevant events, including login attempts, game creations, and access to sensitive functions, creating an audit trail for investigating any security incidents.